Spring 2024

Comprehensive Exam

Yihui (Kyle) Zeng

Tiffany Bao (Chair)

Adam Doupé (Member)

David Brumley (Member)

Fish Wang (Member)

Yan Shoshitaishvili (Member)

$whoami

Mathematics (Bachelor) => Computer Science (PhD)

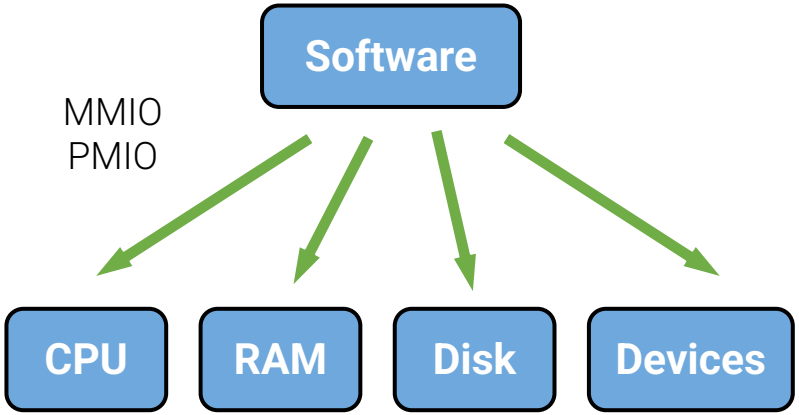
Self-taught hacker

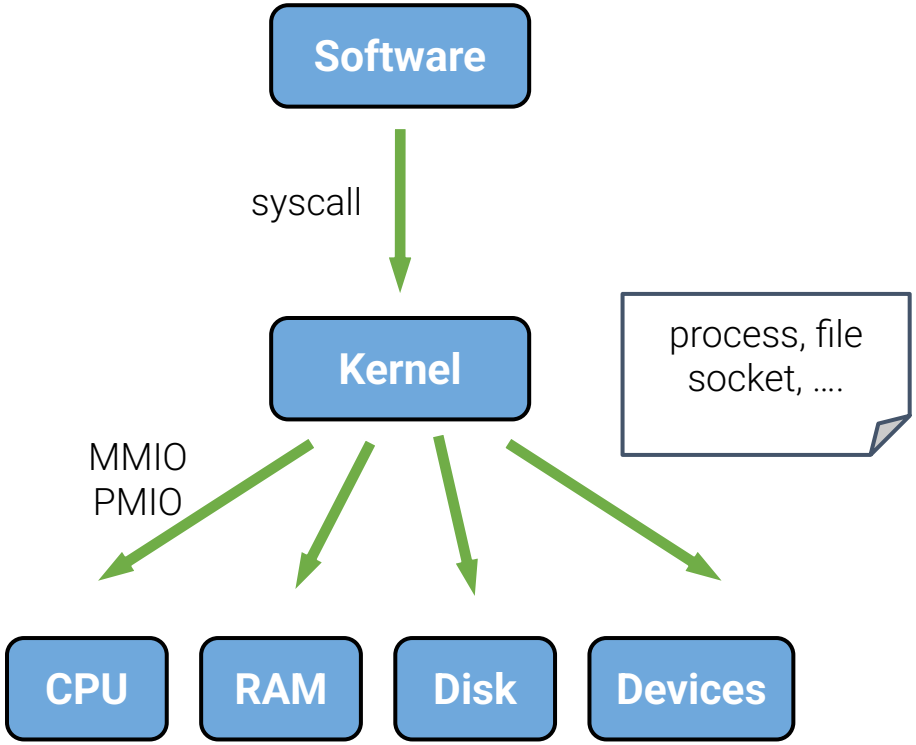
Stumbled upon Linux kernel by accident

Agenda

- Background
- Motivation
- Vulnerability Discovery
- Protections
- Exploitation
- Automated Triage
- Final Thoughts

Background


Software
MMIO
PMIO
CPU
RAM
Disk
Devices


Software
syscall
Kernel
process, file
socket, ...
MMIO
PMIO
CPU
RAM
Disk
Devices

Motivation – cont.

Billions of users

- Android alone: 3.3b

Rise of Linux-based smart home devices

The cloud runs on Linux

Motivation – cont.

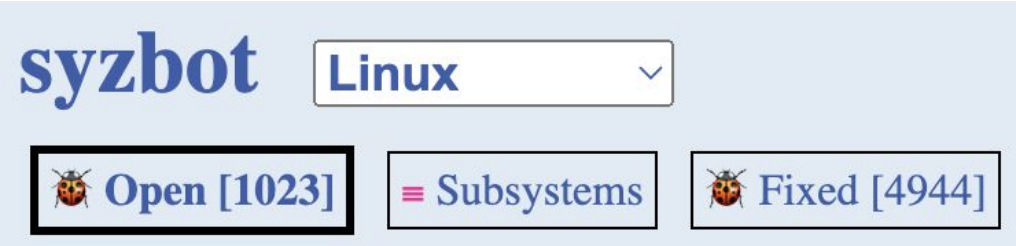
The whole world is aware:

kernelCTF
ChromeOS VRP

Motivation – cont.

Unfortunately, Linux is still *VERY* broken:

Confirmed working exploits in 2023 alone: 69*



* collected by Google's kernelCTF VRP program

Research Is Needed

Vulnerability Discovery

Generic Protections

Exploitation

Automated Triage

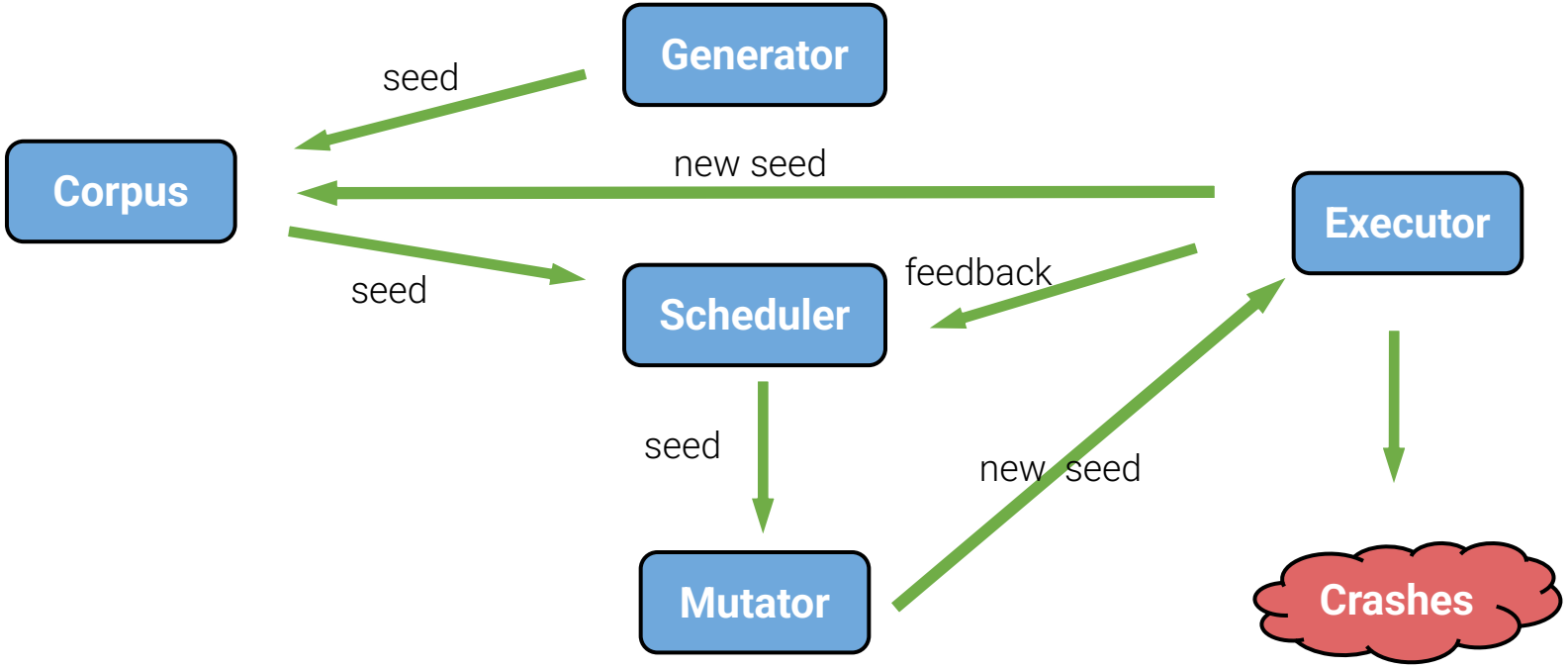
Vulnerability Discovery

Major Research Approaches

Fuzzing

Static Analysis + Manual Verification

Kernel Fuzzing 101



Kernel Fuzzing – Seed

Two dimensional

Type-aware

Resource-aware

```
# https://syzkaller.appspot.com/bug?id=41f079fc054bda515d93a93341f9c4b6cd10614d
# See https://goo.gl/kgGztJ for information about syzkaller reproducers.
#{"procs":1,"sandbox":"namespace","fault_call":-1,"tmpdir":true}
r0 = socket$nl_route(0x10, 0x3, 0x0)
r1 = socket$nl_route(0x10, 0x3, 0x0)
r2 = socket(0x10, 0x803, 0x0)
sendmsg$NBD_CMD_DISCONNECT(r2, &(0x7f0000000280)={0x0, 0x0, &(0x7f0000000180)={0x0
}}, 0x0)
getsockname$packet(r2, &(0x7f0000000100)={0x11, 0x0, <r3=>0x0, 0x1, 0x0, 0x6, @bro
adcast}, &(0x7f0000000200)=0x14)
sendmsg$nl_route_sched(r1, &(0x7f0000000240)={0x0, 0x0, &(0x7f0000000140)={&(0x7f0
0000003c0)=ANY=[@ANYBLOB="380000002400070500000004007a2a30005000000", @ANYRES32=r3,
 @ANYBLOB="00000000ffffffff0000000009000100686673630000000000800020000000000"], 0x3
8}}, 0x0)
sendmsg$nl_route_sched(r0, &(0x7f00000001c0)={0x0, 0x0, &(0x7f0000000180)={&(0x7f0
000000580)=ANY=[@ANYBLOB='D\x00\x00\x00,\x00\'\r\x00'/20, @ANYRES32=r3, @ANYBLOB="
0000000000000000800f1ff0800010075333200180002001400050000000000000000000000000000
7c6724"], 0x44}}, 0x0)
```


* an implementation called syzlang

Kernel Fuzzing – Generator

Syzlang syscall templates

```
resource fd_fscontext[fd]

fsopen(type ptr[in, string[filesystem]], flags flags[fsopen_flags]) fd_fscontext
fspick(dfd fd_dir[opt], path ptr[in, filename], flags flags[fspick_flags]) fd_fscontext
```

Manually written

```
filesystem = "sysfs", "rootfs", "ramfs", "tmpfs", "devtmpfs", "debugfs", "security
fs", "sockfs", "pipefs", "anon_inodefs", "devpts", "hugetlbfs", "vfat", "ecryptfs"
```

Incomplete

FuzzNG: resolve userspace input just-in-time (hooking) to reduce the dependency on syzlang templates

FuzzNG does not fully solve the problem

Kernel Fuzzing – Corpus

Known fact: better initial corpus => better result

Define “better” corpus:

- minimal
- semantic correctness
- traverse deep states

Moonshine: gather traces from real testsuite, dependency tracking, minimization, trace to syzlang

Kernel Fuzzing – Scheduler

Syzkaller’s scheduler is *VERY* bad

SyzVegas: reinforcement learning to schedule

- Seeds
- Fuzzing tasks
    - Generation
    - Mutation
    - Minimization (Triage)

Kernel Fuzzing – Executor

Sanitizer

- KASAN, UBSAN, KMSAN, KCSAN, ……
- UniSan: detect uninitialized info leak to userspace
- Hydra: logic bug checker

Feedback

- kAFL: use intel-PT to obtain code coverage
- KRACE: augment coverage with memory accessing patterns to detect concurrency bugs

Kernel Fuzzing – Mutator

HFL: symbolic execution

DIFUZE: type-aware mutation

Janus: stateful mutation (two dimensional)

Static Analysis

Generic:

- DR. CHECKER: use soundy static analysis for finding bugs
- CRIX: identify missing checks

Kernel-specific:

- CID, LinKRID: find refcount bugs
- Double Fetch Analysis: find double-fetch bugs

Vulnerability Discovery – Trend and Thoughts

Kernel fuzzing is still in early stage

- stateful fuzzing
- scheduling is still in exploration stage
- more customized feedback

Static Analysis

- vulnerability-type-centric
- maybe more fruitful due to constraints on fuzzers

Generic Protections

Generic Protections

User-Kernel Isolation

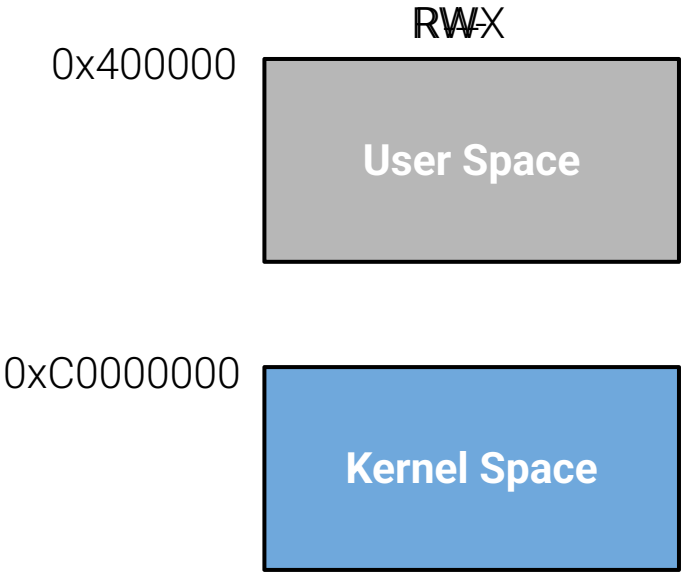
Randomization

Vuln Class Elimination

Kernel->User Isolation

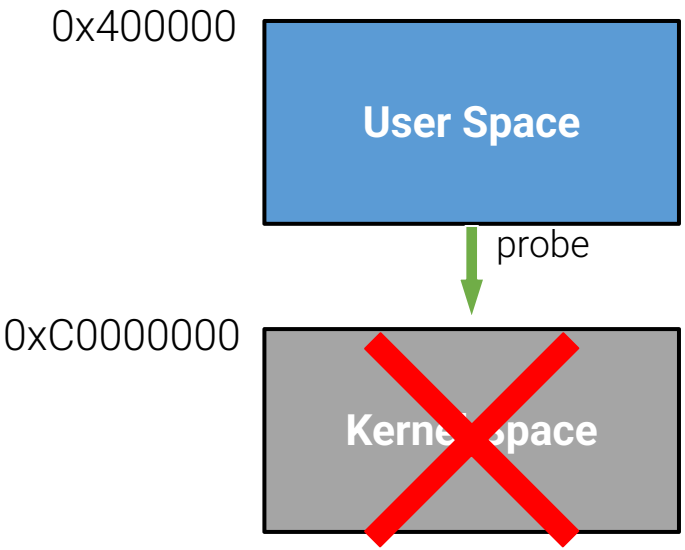
SMEP/PXN

SMAP/PAN



User->Kernel Isolation

KAISER/KPTI



Sensitive-Memory Isolation

xMP: hypervisor-based memory-tagging (domain-tagging) memory isolation

Randomization

KASLR

FG-KALSR/kR^X

0x400000

User Space

0xC000000

Vuln Class Elimination

Midas: mitigate double-fetch vulns

kMVX: eliminate info leak to userspace

pt-rand: prevent page table manipulation

Clang CFI, KCFI, PAL: control-flow hijacking

Generic Protections – Trend and Thoughts

Active but not fruitful

- Performance cost

Utilize external features

- Hardware/hypervisor

Efficacy is questionable

- Most only lengthen exploitation chains

Exploitation

Exploitation Research

Break Isolation

De-Randomization

Data-Only Attack

Sensitive-Object Identification

Exploitability Enhancement

Break ~~Isolation~~

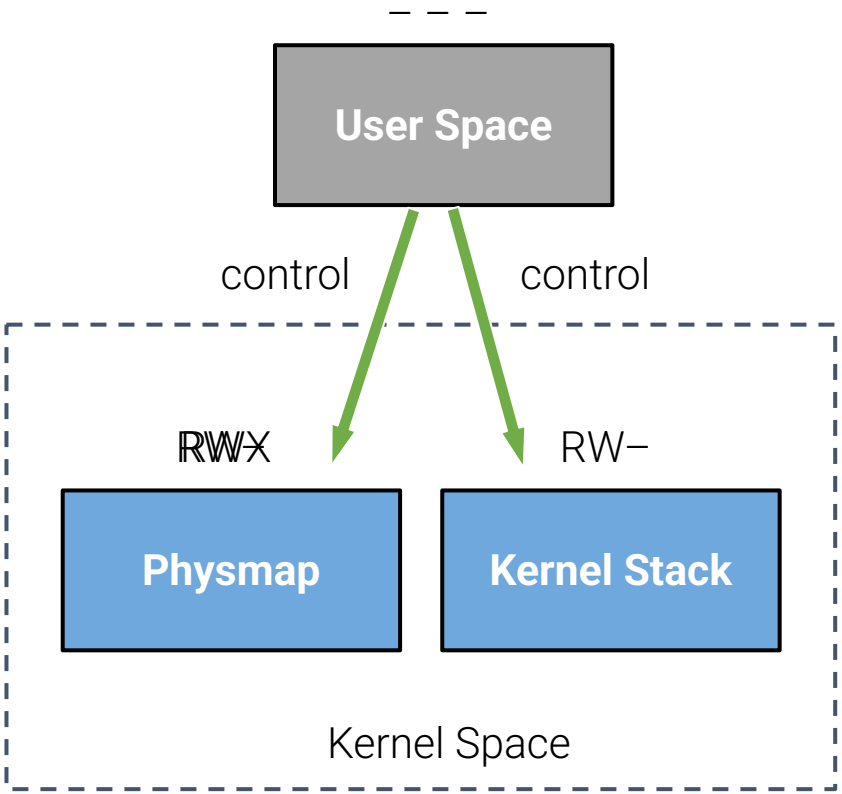
Break Isolation

ret2dir: physmap

ping pong root*: cross-cache attack

KEPLER: heap vuln => stack vuln

RetSpill



* From Collision To Exploitation: Unleashing Use-After-Free Vulnerabilities in Linux Kernel

De-Randomization

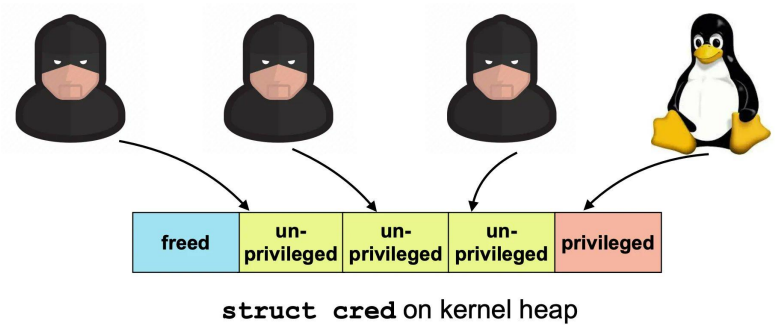
KASLR Bypass

- prefetch, transactional execution: probe whether an address is mapped
- software-based ad-hoc techniques

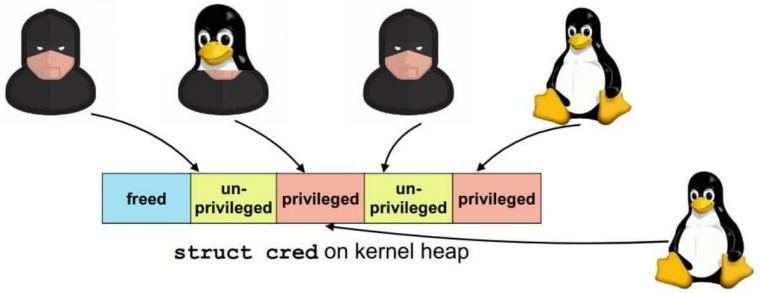
FG-KALSR Bypass:

- blindside: gadget effect analysis using spectre to find gadgets

Data-Only Attacks



DirtyCred: cred swapping



Data-Only Attacks

SCAVY: define security property, randomly corrupt kernel object to detect property-violation

Access to eBPF can break Linux security*

* Cross Container Attacks: The Bewildered eBPF on Clouds

Sensitive Object Identification

SLAKE: static analysis to find object candidates and targeted fuzzing to confirm access and site-of-interest

Eloise: systematically study elastic object

AlphaEXP: over-approximation of sensitive objects using knowledge graph

Exploitability

Targeted Stack Spray: make it possible to exploit use-before-initialization vulnerability reliably

ExpRace: inject interrupts to increase race-window to win races

Exploitation – Trend and Thoughts

User-Kernel Isolation is still not there yet

- Utilize hardware features: MTE, PAC, SGX/TEE

Address Space Layout Randomization is not a powerful defense

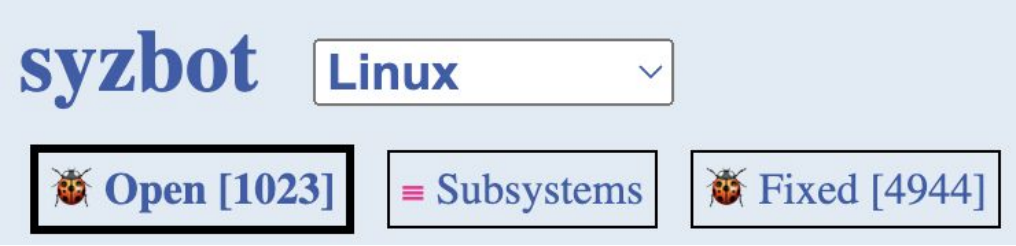
Rise of CFI

- Data-Only Attack may be a fruitful area

Automated Triage

Why Automated Triage

Too many bugs, developers need to prioritize the “severe” ones first



Automated Triage Research Directions

Behavior Analysis

Exploit Generation

Reproduction

Automated Triage – Behavior Analysis

Crash is a symptom - A symptom != Severity of a vuln

SyzScope: symbolize the vulnerable object and symbolically execute all paths that touch the vuln object to identify the worst symptom (with over-approximation)

Grebe: object-driven fuzzing to expose more symptoms

Automated Triage – Exploit Generation

KOOBE: expert system that generates OOB exploits automatically

- Record primitives -> find victim objects -> hardcoded fengshui

FUZE: fuzzing + symbolic execution to generate UAF exploits

- Free -> Fuzzing -> Use -> Symbolic Execution to achieve PC-control

Automated Triage – Reproduction

One upstream kernel, a ton of vendor kernels, a lot of versions

AEM: automatically port exploits across kernel versions by memory access alignment

SyzBridge: systematically analyze the failure factors and port PoCs across kernels by automatically mitigating those factors

Automated Triage – Reproduction

But exploits may be unreliable

K(H)eaps: first work to systematically study the exploit reliability problem

PSPRAY: side-channel kernel heap state to improve exploit reliability

Automated Triage – Trend and Thoughts

We are getting close to such a pipeline

- The community is embracing it
- Effort is needed to build such a pipeline

Hard to triage race bugs

- Hard to reproduce by machines
- Hard to analyze by humans

Final Thoughts

Final Thoughts

- Linux kernel security is important, but it has more bugs than we can fix
  - More research effort should be spent on triaging bugs
- The automated pipeline is coming
- We may want to design better hardware features to build light-weighted protections
- It is getting more and more secure

Thank you!
Q & A

Kyle Zeng

zengyhkyle@asu.edu

@ky1ebot

@Kyle-Kyle